

Where a Hybrid MoE Spends Its Bytes: A 177B Engram Model with a 34.7 GiB GPU-Resident Core

Al-ameen Olajide
Bad Theory Labs

September 2026

Abstract

I compressed Tinfield 1, a fine-tune of the 176.94B-parameter Qwen3.8-Flash-Next model, into two GGUF artifacts. Compact is 72.04 GiB with 45.22 GiB resident on the GPU. Mini is 61.5 GiB with a 34.67 GiB GPU-resident core, 6.8 times smaller than the BF16 GPU allocation. Against an 8-bit build of the same merged weights, mean KL divergence is 0.287 for Compact and 0.340 for Mini on 22 held-out agent-session slices. On a private 120-item free-running behavior gate, Compact retains 98.3% of teacher-correct items and Mini retains 97.4%. The gate is a damage check rather than a quality score; both candidates pass 116 of 120 items in absolute terms.

I measured the architecture and assigned precision by component. Routed experts account for 15.5% of the bytes read by a decoded token, while linear-attention projections account for 40.6% and four-stream hyper-connections for 12.4%. The 51B-parameter n-gram table is highly skewed by access, but pruning 95% of its rows recovers only 28% of the table’s measured output contribution. Simulating a 2.5-bit code across all rows moves mean KL from 0.287 to 0.296, about one reported standard error, and would reduce the table from 26.8 to about 14.9 GiB. For the expert down projections, importance-weighted range search lowers reconstruction error from 51.9% to 11.7% without changing the Q2_0 file layout. Hyper-connections make general rotation edits inexact, while per-channel scale migration remains exact to floating-point precision.

1 Introduction

Qwen3.8-Flash-Next combines sparse experts, linear attention, sparse full attention, four gated residual streams and a 51B-parameter lookup table [1]. Its converted BF16 weights occupy about 330 GiB. Compressing it is therefore an allocation problem across components that have very different storage, access and algebraic properties.

Of its 48 layers, 36 are Gated DeltaNet linear-attention layers and 12 are full attention with a sparse indexer. Every layer reads from and writes to four residual streams through input-dependent hyper-connections. Layer 2 adds rows from a 320-million-row n-gram embedding table that holds 51B of the model’s 177B parameters and is never multiplied.

I measured those components before choosing a quantization recipe. The measurements changed where the bits went:

1. A per-tensor account of decode traffic shows that routed experts are a minority of per-token reads on this architecture (Section 3).
2. On held-out agent sessions, dropping 95% of the engram rows loses most of the table’s measured effect, while lowering precision across all rows changes KL much less (Section 4).

3. Hyper-connections admit exact per-channel scale migration but block general rotation-based outlier removal (Section 5).
4. Importance-weighted range search makes 2-bit expert down projections viable in the existing Q2_0 layout, reducing the GPU-resident core from 45.22 to 34.67 GiB (Section 7).

2 The model

Layers	48: 36 Gated DeltaNet, 12 full attention (every fourth layer)
Residual	4 hyper-connection streams, low rank 320, input-dependent gates
Experts	512 routed per layer, top-10, width 640, plus one shared expert
Full attention	2 KV heads, head dim 256, partial RoPE (64 of 256 dims), sparse indexer
Engram table	320,001,536 rows $\times$ 160 dims, 16 hash heads (8 bigram, 8 trigram), read at layer 2
Vocabulary	248,320, untied input and output embeddings
Parameters	176.94B in the converted model (the one-layer MTP head is not converted)

Table 1: Qwen3.8-Flash-Next (revision f5d0827), the base of Tinfield 1 [1].

Tinfield 1 is this model with a LoRA adapter merged in, trained on 114 reviewed agentic sessions (2.94M tokens). The adapter is not the subject of this paper. Every retention number below compares the compressed model with an 8-bit build of the same merged weights, so it measures what compression removed, not what fine-tuning changed.

3 Where the bytes go

Stored size and decode traffic rank the components differently. In BF16, routed experts take about 240 GB, the engram table about 102 GB and everything else about 15 GB. A decoded token, however, reads only 10 of 512 experts per layer and a few engram rows, while it reads every dense projection in full.

I summed per-tensor sizes from the quantizer’s dry run of the final build (Section 6) and weighted each tensor by the fraction read per token. A token reads about 5.09 GiB:

Component	Share of per-token reads
Linear-attention projections (36 layers)	40.6%
Routed experts (10 of 512 per layer)	15.5%
Hyper-connections (read and write, every layer)	12.4%
Full attention and sparse indexer (12 layers)	12.0%
Output head (248,320 rows)	9.5%
Routers (kept at F32 by llama.cpp)	4.6%
Shared expert	4.6%

Table 2: Decode traffic of the compressed build. Experts are quantized to 2 to 4 bits here, which lowers their share; at uniform precision they would still be a minority of reads.

If decode is bandwidth-bound, speed follows Table 2, not the storage breakdown. The dense spine is about 80% of the modeled traffic, so I kept it on the GPU and treated the experts as the first offload candidate. The measured B200 configuration in Section 6 did not reach the bandwidth ceiling, so this accounting is an allocation model rather than a throughput prediction.

4 The engram table: row coverage over row precision

The engram table maps hashed bigrams and trigrams of the input to 160-dimensional rows that layer 2 adds to the residual [2, 1]. I first looked at it as weights. No row has a norm below 10% of the median, and rows are not low rank (128 of 160 dimensions keep 83% of the energy). Those statistics offered no obvious pruning rule.

I then reimplemented the model’s hash exactly (validated against the reference implementation) and ran it over 912 rendered agentic sessions, 742 for fitting and 170 held out, 10.9M tokens in all. The fit split touches only 6.8% of the table. Choosing the most-read rows per head on the fit split and measuring on held-out sessions:

Hottest rows kept	Size at 8 bits	Held-out reads served	Fit reads served
0.1%	0.05 GB	50.9%	55.1%
0.5%	0.27 GB	60.8%	70.3%
1%	0.54 GB	64.9%	77.7%
5%	2.72 GB	73.9%	95.0%
10%	5.44 GB	76.7%	100%

Table 3: Engram access skew. The full table is 54.4 GB at 8 bits.

Bigram heads are more concentrated than trigram heads: the hottest 0.1% of a bigram head serves 60% of its held-out reads, against 42% for a trigram head. The eight heads of each kind behave almost identically, which is what independent hash functions over the same n-grams should produce.

The last column matters as much as the others. 23.9% of held-out reads land on rows the fit split never touched, so no hot set chosen in advance serves more than about three quarters of reads. Whether those cold reads matter is a question about the model’s output, not about the table.

Ablation. I expected the skew to mean the tail could go. To test that, I edited the engram table of the compressed build and measured KL divergence against the 8-bit teacher (which keeps its full table) on the held-out text. Hot rows were chosen per head on the fit split, which the evaluation text never contains. All runs use the same text, context length (4,096) and KV cache setting, and the rest of the model is identical.

Lower precision was tested inside the shipped IQ4_NL container by restricting each block’s 4-bit indices to a subset of the format’s 16-level grid: 8 levels for a 3-bit code and 4 levels for a 2-bit code, with each block’s scale re-chosen by a search over clip ratios and both signs to minimize squared error. With the fp16 block scale these codes cost 3.5 and 2.5 bits per weight. llama.cpp runs the edited file unchanged, so the measured KL is exactly that of the restricted code; only the file size is not yet reduced. Before writing, the codec’s decoder was checked to agree exactly with llama.cpp’s reference on real blocks. The codes were re-encoded from the 4.5-bit table rather than from BF16, which can only make them look worse.

The table matters: removing it adds 0.192 to mean KL and costs 4.2 points of top-token agreement. Keeping the hottest 5%, which serve 73.9% of held-out reads, removes only 0.053 of that 0.192, or 28%. The remaining 72% of the table’s effect lives in rows that are read about a quarter of the time.

The fill value for dropped rows explains one anomaly and nothing else. With zeroed rows, perplexity is worse than with no table at all (1.158 against 1.137), which suggests a zero row is an input the model never saw in training. Replacing each dropped row with its head’s mean row

Engram table	Rows	Bits	Mean KL	Same top token	PPL / teacher
Full (shipped)	320.0M	4.5	0.287 ± 0.008	87.6%	0.972
All rows, 3-bit code	320.0M	3.5	0.306 ± 0.009	87.4%	0.977
All rows, 2-bit code	320.0M	2.5	0.296 ± 0.008	87.3%	0.984
Hot 5% at 4.5, rest 2-bit	320.0M	2.6	0.293 ± 0.008	87.3%	0.994
Hot 5%, rest zero	16.0M	4.5	0.426 ± 0.010	84.7%	1.158
Hot 5%, rest mean row per head	16.0M	4.5	0.432 ± 0.011	84.9%	1.053
Removed	0	–	0.479 ± 0.011	83.4%	1.137

Table 4: Engram ablations on the compressed build, KL against the Q8_0 teacher with its full table. Error bars are llama.cpp’s reported standard errors of the mean.

fixes perplexity (1.053) but leaves KL where it was (0.432 against 0.426). A mean row cannot carry what a specific rare n-gram’s row knows, and that information is what the cold rows contribute.

Precision is a different matter. At 2.5 bits per weight the table’s rows carry about 10% relative reconstruction error, yet mean KL moves from 0.287 to 0.296, about one standard error, and top-token agreement stays within 0.3 points. The 3-bit code lands at 0.306, about two standard errors from the shipped table and not distinguishable from the 2-bit code; the ordering between them is noise. Keeping the hot rows at 4.5 bits on top of that buys nothing measurable. In this experiment, broad row coverage mattered more than fine precision within each row.

For this workload and the tested cutoff, lowering precision across all rows was much less damaging than pruning rows. A 2.5-bit engram table would be about 14.9 GiB instead of 26.8 GiB, bringing Compact’s whole file from 72.04 to about 60.1 GiB. Realizing that saving requires a 2-bit block type with 32-value blocks: stock llama.cpp’s sub-4-bit types use blocks of 64 or 256 values, while the table’s rows are 160 wide. I added IQ2_NL, which stores one fp16 scale and 2-bit indices per 32 values. In llama.cpp’s quantization tests its error is about half of Q2_0’s (0.0045 against 0.0087), its encoder matches the codec behind Table 4 on 99.995% of blocks, and all 231 Metal GET_ROWS tests pass against the CPU reference. The CUDA row-lookup path is written but has not been compiled or run, and IQ2_NL has not yet been tested in a full model.

5 Hyper-connections and exact weight edits

Scale migration moves quantization difficulty from one tensor to another without changing the function [3, 4]. Rotation methods [5, 6] go further and spread outliers across channels with an orthogonal matrix. Both assume that the tensors between the two edited weights act linearly or channel-wise in a compatible way. Hyper-connections [7] test that assumption.

The hyper-connection read. For streams $x_1, \dots, x_4$, the block input is

$$m = \frac{1}{4} \sum_k g_k \odot n_k, \quad n_k = \text{RMSNorm}(x_k) \odot w, \quad g_k = \sigma(U \text{silu}(D[n_1; \dots; n_4]))_k,$$

and the block’s first projection computes Wm . The injection gate on the way out is

$$2\sigma(J[n_1; \dots; n_4]).$$

Exact: per-channel scaling. Replace w by w/s for a positive vector s . The RMS denominator depends only on x_k , so n_k becomes n_k/s . Replacing D by $D \text{diag}(\text{tile}(s))$ and J by $J \text{diag}(\text{tile}(s))$

restores every gate exactly, so m becomes m/s , and $W \text{diag}(s)$ restores the block. Inside a SwiGLU expert, $\text{down}(\text{silu}(Gx) \odot Ux)$ is unchanged by $U \rightarrow \text{diag}(1/s)U$, $\text{down} \rightarrow \text{down} \text{diag}(s)$ because the product is element-wise.

Not exact: rotation. Rotating the block input requires $Q^\top(g \odot n) = g' \odot Q^\top n$ for some gate g' . For a general orthogonal Q no such g' exists, because g scales each feature independently and depends on the input. Only signed permutations commute with the gate.

I verified all three statements in float64 on a replica of the block math, with random weights at the model’s structure:

Edit	Max relative output error
A. Expert scale migration (SwiGLU)	9.9×10^{-17}
B. Scale migration through the hyper-connection read	9.9×10^{-17}
C. Rotation of the block input (QuaRot family)	1.04×10^{-2}

I also prepared the scale migration for the real expert tensors using $s = \mathbb{E}[x^2]^{\alpha/2}$ with $\alpha = 0.5$ and adjusted the importance matrix for the rescaled inputs. The median layer’s 5th and 95th percentile scales are 0.82 and 1.31. I do not report a quality result for this edit because the requantized build has not been measured.

6 The build

Recipe. I converted the merged model with llama.cpp and quantized it with an importance matrix computed on 41K tokens of agentic text: expert gate and up projections at IQ2_XXS, expert down projections at IQ4_NL (their width of 640 is not a multiple of 256, which rules out the 2-bit K-quant and IQ2 formats; Section 7 uses Q2_0, whose 64-value blocks do fit), the engram table at IQ4_NL, the token embedding and output head at Q6_K, and the rest of the spine at Q8_0. The routers stay at F32.

Merge integrity. Before conversion, I verified all 131 source shards. Every one of the 530 LoRA targets in the final merge equals the base tensor plus $(BA) \times 2.0$ at the stored dtype, every non-target tensor is unchanged, and tensor names, shapes and dtypes match the base.

Size. The file is 72.04 GiB, 3.50 bits per weight on average. The GPU-resident part (everything except the engram table, which is memory-mapped on the CPU side) is 45.22 GiB: experts 40.43 GiB and spine 4.8 GiB, 3.1 bits per weight. Against BF16 (176.94B parameters, 330 GiB), the file is 4.6 times smaller and the GPU-resident part 5.2 times smaller.

KL divergence. On 22 held-out session slices, with the KV cache at 8-bit keys and 4-bit values (9,984 bytes per token instead of 24,576), against the Q8_0 build of the same model: mean KL 0.287, median 0.016, 99th percentile 4.29, same top token 87.6%, perplexity ratio 0.972.

Behavior. KL divergence cannot certify that a compressed model still behaves like its teacher; a few high-KL tokens can flip an answer, and many can leave it unchanged. I therefore ran a private free-running gate at temperature 0 with the teacher and the candidate served identically:

Item type	Retained (of teacher-correct)
Confident recall	39 / 39
False-premise pushback	36 / 38
Grounded extraction	40 / 40
Tool-call probes, identical calls	9 / 10
Gate retention	115 / 117 (98.3%)

The one tool-call miss is the case agents care about most: asked for the weather in two cities, the candidate called the tool once.

Speed. On one B200 with llama.cpp, generation runs at 96 tokens per second with every expert on the GPU, 17.6 with half the expert layers in CPU memory, and 13 with all experts in CPU memory (prompt processing 2,273, 347 and 206 tokens per second). The last configuration needs only the 4.8 GiB spine and the KV cache on the GPU. These figures describe this configuration, not the architecture: a decoded token reads about 5.09 GiB, so a bandwidth-bound run on this card would be near 1,460 tokens per second and 96 is 6.6% of that. The limit is not memory bandwidth and I have not profiled it, so the numbers should not be scaled to other hardware.

7 A smaller build: range-searched 2-bit down projections

The expert down projections hold a third of the expert weights and were the one expert tensor left at 4.5 bits. llama.cpp’s Q2_0 uses 64-value blocks, so it fits their 640-wide rows, but its quantizer sets each block’s scale to the block maximum. With levels at -1 , 0 , $+1$ and $+2$ times that scale, most weights round to zero, the $+2$ level is never used, and the importance matrix is ignored. On the real down projections this rule leaves 51.9% imatrix-weighted relative error.

I kept llama.cpp’s file layout and replaced the encoding. For every block, the encoder tries 17 clip ratios (0.2 to 1.0 of the block maximum) and both signs of the scale, and keeps the one with the lowest squared error weighted by the per-expert input energy from the importance matrix. That lowers the error to 11.7%. Before writing, the encoder reproduces llama.cpp’s own rule on real blocks (all scales and 99.97% of bytes identical; the rest are rounding ties), so the layout is verified rather than assumed.

	Compact	Mini
Whole file	72.04 GiB	61.5 GiB
Smaller than BF16 (whole file)	4.6×	5.4×
GPU-resident size	45.22 GiB	34.67 GiB
Smaller than BF16 (GPU side)	5.2×	6.8×
Mean KL vs Q8_0	0.287	0.340
Gate retention	98.3%	97.4%
Tool probes, identical calls	9/10	6/10
Tool probes, same calls ignoring optional arguments	9/10	8/10

Table 5: The 2-bit down-projection build against the 4.5-bit one. Both candidates pass 116 of 120 gate items in absolute terms; the conditional-retention difference is one teacher-correct item.

Exact tool-call matching overstates part of the difference: in four probes Mini adds an optional

`unit` argument. Under a functional comparison that ignores optional arguments, Mini matches 8 of 10 calls. The remaining misses are both parallel requests in which it emits only one of two required calls. The private gate and probe set are small damage checks, and the one-item retention difference does not establish an ordering between the builds.

The range search now lives in `llama.cpp`'s own `Q2_0` quantizer whenever an importance matrix is supplied, byte-identical to the encoder used for this build, so the file can be rebuilt with `llama-quantize` alone.

8 What this work contributes

The contribution is the measured allocation and its implementation. The paper accounts for bytes read per decoded token instead of treating stored parameter count as runtime cost. It separates engram access frequency from engram output effect on held-out sessions. It proves which channel transforms commute with the model's hyper-connections, and it carries an importance-weighted range search into `llama.cpp` without changing `Q2_0`'s on-disk representation. `IQ2_NL` extends the same restricted-grid experiment to 160-wide lookup rows with a tested CPU and Metal path.

This work does not claim to invent mixture-of-experts models, Engram, importance matrices, block clipping, low-bit GGUF formats, rotation methods or scale migration. Its claim is narrower: measuring this hybrid architecture changes the compression decision, and those measurements produced two complete artifacts with 45.22 and 34.67 GiB GPU-resident cores.

9 Limitations

The access statistics come from one distribution of agentic sessions; other workloads may have different hot sets. The expert routing skew I measured is in-sample on 41K tokens and is not treated as a result. The KL and gate figures compare against an 8-bit build, not BF16. KL measures distributional fidelity, not downstream quality. The engram ablations test one pruning cutoff (5%) and two fill values (zero and the per-head mean); cutoffs between 5% and 100% and learned fallbacks remain untested. The low-precision engram codes are simulated inside the 4.5-bit container, so their smaller file sizes are derived rather than measured. `IQ2_NL` has passed CPU and Metal row-lookup tests but not CUDA or a full-model run. The scale-migration proof establishes functional equivalence of the edit in float64, not improved quantization after the edit. Finally, the B200 speed figures describe one unprofiled `llama.cpp` configuration and should not be scaled to other hardware.

10 Related work

Qwen3.8-Flash-Next and its gated residual, sparse attention and off-accelerator n-gram memory are described by Qiu et al. [1]. Engram originates with Cheng et al.'s conditional-memory work [2]. Scale migration comes from SmoothQuant [3] and AWQ [4]; rotation-based quantization from QuaRot [5] and SpinQuant [6]. Hyper-connections were introduced by Zhu et al. [7]; Gated DeltaNet by Yang et al. [8]. TEAL [9] exploits activation sparsity in dense projections, the component class that dominates the byte model here. Converting trained attention to a latent KV cache (MHA2MLA [10], TransMLA [11]) needs further training; a weight-space SVD of this model's key and value projections gave no training-free margin, so I did not pursue it.

11 Conclusion

The smallest artifact in this study stores a 176.94B-parameter hybrid model in 61.5 GiB and leaves 34.67 GiB resident on the GPU. It came from three architecture-specific decisions: keep the dense spine resident because it dominates per-token reads, compress the engram table by precision rather than by the tested row-pruning route, and search the representable range of 2-bit expert blocks instead of accepting the block maximum. The result is not a universal quantization recipe. It is a complete byte allocation for one unusual architecture, backed by held-out KL, exact edit tests, encoder cross-checks and two runnable GGUF artifacts.

References

- [1] Z. Qiu et al. On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability. arXiv:2608.30320, 2026.
- [2] X. Cheng et al. Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models. arXiv:2601.07372, 2026.
- [3] G. Xiao et al. SmoothQuant: Accurate and efficient post-training quantization for large language models. arXiv:2211.10438, 2022.
- [4] J. Lin et al. AWQ: Activation-aware weight quantization for LLM compression and acceleration. arXiv:2306.00978, 2023.
- [5] S. Ashkboos et al. QuaRot: Outlier-free 4-bit inference in rotated LLMs. arXiv:2404.00456, 2024.
- [6] Z. Liu et al. SpinQuant: LLM quantization with learned rotations. arXiv:2405.16406, 2024.
- [7] D. Zhu et al. Hyper-connections. arXiv:2409.19606, 2024.
- [8] S. Yang, J. Kautz, A. Hatamizadeh. Gated Delta Networks: Improving Mamba2 with delta rule. arXiv:2412.06464, 2024.
- [9] J. Liu et al. Training-free activation sparsity in large language models. arXiv:2408.14690, 2024.
- [10] T. Ji et al. Towards economical inference: Enabling DeepSeek’s multi-head latent attention in any Transformer-based LLMs. arXiv:2502.14837, 2025.
- [11] F. Meng et al. TransMLA: Multi-head latent attention is all you need. arXiv:2502.07864, 2025.